

DTB Clase 8 - Lab FabCar

Objetivo

Implementar, desplegar y probar el contrato inteligente **fabcar** utilizando JavaScript y el Hyperledger Fabric Debugger en Visual Studio Code. Esta actividad te permitirá familiarizarte con el desarrollo de contratos inteligentes en Hyperledger Fabric, así como con el uso de herramientas de depuración para mejorar la eficiencia y precisión del desarrollo.

Pre-requisitos

- Docker instalados
- Node.js y npm instalados
- Visual Studio Code instalado
- Familiaridad con conceptos básicos de blockchain y Hyperledger Fabric

Hyperledger Fabric Debugger

El Hyperledger Fabric Debugger es una extensión para Visual Studio Code que facilita la creación, despliegue y depuración de contratos inteligentes en Hyperledger Fabric. Esta herramienta automatiza la configuración de una red local de Hyperledger Fabric, lo que permite a los desarrolladores centrarse en el desarrollo y la prueba de su código de chaincode. Entre sus características destacadas se incluyen:

- **Gestión Automática de Redes Locales:** Configura una red local de Fabric con un solo clic, incluyendo nodos peer, orderer, CouchDB, y CA.
- **Depuración Integrada:** Permite configurar puntos de interrupción, inspeccionar variables, y rastrear la ejecución del chaincode directamente desde Visual Studio Code.
- **Prueba de Transacciones:** Permite enviar transacciones de prueba directamente desde el editor para verificar el funcionamiento del chaincode.
- **Persistencia de Datos:** La red local conserva los datos del ledger incluso después de reiniciar los contenedores de Docker, lo que facilita pruebas continuas y repetidas.

Estructura del Activo

Estructura del Activo **Car**:

```
{
  "Key": "CAR0",
  "Record": {
    "docType": "car",
    "color": "blue",
    "make": "Toyota",
    "model": "Prius",
    "owner": "Tomoko"
  }
}
```

Cada automóvil en el ledger tiene la siguiente estructura:

- **Key:** Identificador único del automóvil (e.g., "CAR0").
- **Record:**
 - **docType:** Tipo de documento, en este caso, siempre "car".
 - **color:** Color del automóvil.
 - **make:** Marca del automóvil.
 - **model:** Modelo del automóvil.
 - **owner:** Propietario del automóvil.

Operaciones del Chaincode

1. **initLedger:** Inicializa el ledger con una lista de automóviles predefinidos.
2. **queryCar:** Consulta un automóvil específico por su ID.
3. **createCar:** Crea un nuevo automóvil en el ledger.
4. **queryAllCars:** Consulta todos los automóviles en el ledger.
5. **changeCarOwner:** Cambia el propietario de un automóvil específico.

Pasos

Paso 1: Configuración del Entorno

1. **Instalar la Extensión de Hyperledger Fabric Debugger:**
 - Abre Visual Studio Code.
 - Ve a la pestaña de Extensiones y busca "Hyperledger Fabric Debugger".
 - Instala la extensión.
2. **Iniciar Docker.**
3. **Iniciar la Red Local con Hyperledger Fabric Debugger:**
 - Utiliza el comando de la extensión para iniciar la red local. Esto creará automáticamente una red de Hyperledger Fabric con un solo peer, un orderer, CouchDB, y un CA (Certificate Authority).

Paso 2: Desarrollar el Contrato Inteligente en JavaScript

1. Abrir un Terminal (Linux / Mac) o PowerShell (Windows) para crear el Directorio del Chaincode con los siguientes comandos:

```
mkdir chaincode-fabcar  
cd chaincode-fabcar
```

1. Inicializar un Nuevo Proyecto Node.js:

```
npm init -y
```

2. Instalar Dependencias de Hyperledger Fabric:

```
npm install fabric-shim fabric-contract-api
```

3. Abrimos el proyecto en VSCode:

```
code .
```

4. Creamos el Archivo de Chaincode **fabcar.js**:

```
'use strict';  
  
const { Contract } = require('fabric-contract-api');  
  
class FabCar extends Contract {  
  
  async initLedger(ctx) {  
    console.info('===== START : Initialize Ledger =====');  
    const cars = [  
      {  
        make: 'Toyota',  
        model: 'Prius',  
      },  
    ],  
  },  
}
```

```

        color: 'blue',
        owner: 'Tomoko',
    },
    {
        make: 'Ford',
        model: 'Mustang',
        color: 'red',
        owner: 'Brad',
    },
    // Agrega más autos según sea necesario
];

for (let i = 0; i < cars.length; i++) {
    cars[i].docType = 'car';
    await ctx.stub.putState('CAR' + i,
Buffer.from(JSON.stringify(cars[i])));
    console.info('Added <--> ', cars[i]);
}
console.info('===== END : Initialize Ledger =====');
}

async queryCar(ctx, carNumber) {
    const carAsBytes = await ctx.stub.getState(carNumber);
    if (!carAsBytes || carAsBytes.length === 0) {
        throw new Error(`${carNumber} does not exist`);
    }
    console.log(carAsBytes.toString());
    return carAsBytes.toString();
}

async createCar(ctx, carNumber, make, model, color, owner) {
    console.info('===== START : Create Car =====');
    const car = {
        docType: 'car',
        make,
        model,
        color,
        owner,
    };
};

```

```

        await ctx.stub.putState(carNumber,
Buffer.from(JSON.stringify(car)));
        console.info('===== END : Create Car =====');
    }

    async queryAllCars(ctx) {
        const startKey = 'CAR0';
        const endKey = 'CAR999';
        const iterator = await ctx.stub.getStateByRange(startKey, endKey);

        const allResults = [];
        while (true) {
            const res = await iterator.next();
            if (res.value && res.value.value.toString()) {
                console.log(res.value.value.toString());
                allResults.push(res.value.value.toString());
            }
            if (res.done) {
                console.log('end of data');
                await iterator.close();
                return allResults;
            }
        }
    }

    async changeCarOwner(ctx, carNumber, newOwner) {
        console.info('===== START : Change Car Owner =====');
        const carAsBytes = await ctx.stub.getState(carNumber);
        if (!carAsBytes || carAsBytes.length === 0) {
            throw new Error(`${carNumber} does not exist`);
        }
        const car = JSON.parse(carAsBytes.toString());
        car.owner = newOwner;

        await ctx.stub.putState(carNumber,
Buffer.from(JSON.stringify(car)));
        console.info('===== END : Change Car Owner =====');
    }

    async GetCarHistory(ctx, carNumber) {

```

```

        let resultsIterator = await ctx.stub.getHistoryForKey(carNumber);
        let results = await this._GetAllResults(resultsIterator, true);

        return JSON.stringify(results);
    }

    async _GetAllResults(iterator, parse) {
        const allResults = [];
        while (true) {
            const res = await iterator.next();
            if (res.value && res.value.value.toString()) {
                const result = parse
                    ? JSON.parse(res.value.value.toString())
                    : res.value.value.toString();
                allResults.push(result);
            }
            if (res.done) {
                await iterator.close();
                return allResults;
            }
        }
    }
}

module.exports = FabCar;

```

5. Creamos el archivo **index.js** con el siguiente código:

```

'use strict';

const FabCar = require('./fabcar');

module.exports.FabCar = FabCar;

```

```
module.exports.contracts = [FabCar];
```

Paso 3: Depuración del Chaincode

1. Configurar el Debugger:

- Creamos un archivo `.vscode/launch.json` en tu proyecto con la siguiente configuración:

```
{
  "configurations": [
    {
      "name": "Debug Chaincode",
      "type": "hlf-node",
      "request": "launch",
      "isCaas": false
    }
  ]
}
```

2. Iniciar la Depuración:

- Presiona **F5** en Visual Studio Code para iniciar la depuración del chaincode.
- Utiliza los puntos de interrupción y otras herramientas de depuración para inspeccionar y probar tu chaincode.

Paso 4: Probar el Chaincode con Hyperledger Fabric Debugger

1. Crear un Archivo `.fabric` para Probar Transacciones:

Crea un archivo `test.fabric` en la raíz del proyecto con el siguiente contenido:

```
[
  {
    "invoke": "initLedger",
    "args": []
  },
  {
    "query": "queryAllCars",
    "args": []
  }
]
```

```
    },
    {
      "invoke": "createCar",
      "args": [ "CAR12", "Honda", "Civic", "black", "Dave" ]
    },
    {
      "query": "queryCar",
      "args": [ "CAR12" ]
    },
    {
      "invoke": "changeCarOwner",
      "args": [ "CAR12", "Alice" ]
    },
    {
      "query": "GetCarHistory",
      "args": [ "CAR12" ]
    }
  ]
}
```

2. Ejecutar Transacciones desde Visual Studio Code:

- Abre `test.fabric` en Visual Studio Code.
- Verás enlaces "Send Request" junto a cada transacción.
- Haz clic en "Send Request" para ejecutar cada transacción y ver el resultado en la consola de salida.

3. Verificar Salidas y Eventos:

- Asegúrate de que los resultados de las transacciones se muestren en la consola de salida.
- Los eventos emitidos por el chaincode también deberían aparecer en la consola si están configurados correctamente.